

Responsive Layouts in Flet

From Fixed-Pixel Canvases to Fluid Grid Architectures

Overview

A surprising number of desktop and embedded GUI applications are still built like print layouts: every widget is placed at an absolute (x, y) coordinate, tuned to look perfect on the developer's monitor and broken everywhere else. Flet, following Flutter's layout model, replaces absolute positioning with a **constraint-based, nested-container** system built around two primitives: `ft.Row` and `ft.Column`. Mastering how these compose — along with alignment and expansion properties — is the difference between a UI that "looks fine on my screen" and one that genuinely adapts to any viewport.

1 The Situation: Absolute Positioning Doesn't Scale

Hardcoding pixel coordinates or fixed widths bakes a single screen size into the application:

```
1 import flet as ft
2
3 def main(page: ft.Page):
4     # Anti-pattern: fixed positions and sizes
5     title = ft.Text("Dashboard", left=20, top=20)
6     sidebar = ft.Container(width=250, height=800, bgcolor="#222222")
7     content = ft.Container(width=950, height=800, bgcolor="#333333")
8
9     page.add(ft.Stack([title, sidebar, content]))
10
11 ft.app(target=main)
```

On the developer's 1920px-wide monitor, the sidebar and content might fit perfectly. On a 1366px laptop, a 4K display, or a docked panel inside a larger application, the same layout either overflows, leaves dead space, or clips entirely. The root problem is that **nothing in this layout knows how to share space** — every dimension is a hardcoded constant.

2 The Solution: Rows, Columns, and Nested Containers

Flet's layout system is built on two axis-aligned primitives:

- `ft.Row` arranges children horizontally.
- `ft.Column` arranges children vertically.

Each child can declare how it wants to share the available space along that axis — this is the foundation of a fluid, monitor-agnostic UI.

2.1 The expand Property

The single most important property for responsive Flet layouts is **expand**. Setting **expand=True** (or an integer weight) tells a control to grow and fill the remaining space along its parent's main axis, proportionally to other expanding siblings:

```
1 import flet as ft
2
3 def main(page: ft.Page):
4     page.title = "Adaptive Dashboard"
5
6     sidebar = ft.Container(
7         content=ft.Text("Sidebar"),
8         bgcolor="#1f2933",
9         expand=1,      # takes 1 share of the row's width
10    )
11
12    content = ft.Container(
13        content=ft.Text("Main Content"),
14        bgcolor="#323f4b",
15        expand=4,      # takes 4 shares -- i.e. 4x wider than the
                       # sidebar
16    )
17
18    page.add(
19        ft.Row(
20            controls=[sidebar, content],
21            expand=True,  # the row itself fills the page
22        )
23    )
24
25 ft.app(target=main)
```

Because the split is expressed as a *ratio* (1 share vs. 4 shares) rather than a pixel count, this layout automatically rebalances whether the window is 800px or 3840px wide.

2.2 Nesting Rows Inside Columns (and Vice Versa)

Real interfaces are grids, not single strips. The grid emerges naturally by nesting **ft.Row** and **ft.Column** inside each other:

```
1 import flet as ft
2
3 def main(page: ft.Page):
4     page.title = "Terminal Workspace Grid"
5
6     header = ft.Container(
7         content=ft.Text("System Status", size=20, weight=ft.
8             FontWeight.BOLD),
9         bgcolor="#1f2933", padding=10, expand=False,
10    )
11
12    sidebar = ft.Container(
13        content=ft.Column([
14            ft.Text("Navigation"),
15            ft.Text("Logs"),
16            ft.Text("Settings"),
17        ]),
18        bgcolor="#27313a", padding=10, expand=1,
```

```

18     )
19
20     main_panel = ft.Container(
21         content=ft.Text("Live Output Stream"),
22         bgcolor="#323f4b", padding=10, expand=3,
23     )
24
25     body = ft.Row(
26         controls=[sidebar, main_panel],
27         expand=True,
28     )
29
30     page.add(
31         ft.Column(
32             controls=[header, body],
33             expand=True,
34             spacing=0,
35         )
36     )
37
38     ft.app(target=main)

```

Here, a **Column** stacks a fixed-height header above a fluid body; the body is itself a **Row** that splits horizontally between a sidebar and a main panel — a two-dimensional grid built entirely from one-dimensional primitives.

2.3 Alignment Properties

Within a **Row** or **Column**, two alignment axes control how children are positioned when they don't fill all available space:

- **alignment** — distribution along the *main* axis (e.g. `ft.MainAxisAlignment.SPACE_BETWEEN`, `ft.MainAxisAlignment.CENTER`).
- **vertical_alignment** (in a **Row**) / **horizontal_alignment** (in a **Column**) — alignment along the *cross* axis (e.g. `ft.CrossAxisAlignment.STRETCH`).

```

1 toolbar = ft.Row(
2     controls=[
3         ft.Text("Status: Online"),
4         ft.IconButton(icon=ft.icons.REFRESH),
5     ],
6     alignment=ft.MainAxisAlignment.SPACE_BETWEEN,    # push apart on
7     vertical_alignment=ft.CrossAxisAlignment.CENTER, # center on the
8 )

```

`CrossAxisAlignment.STRETCH` is particularly useful for grid-like layouts: it forces every child to occupy the full height (in a **Row**) or full width (in a **Column**) of the cross axis, ensuring panels line up cleanly even as the window resizes.

3 The Outcome: A Fluid Interface Matrix

Combining **expand** ratios, nested **Row/Column** containers, and alignment properties produces a layout that behaves like a true grid — one that redistributes space proportionally rather than clipping or leaving gaps.

```

1 import flet as ft
2
3 def main(page: ft.Page):
4     page.title = "Fluid Terminal Matrix"
5     page.padding = 0
6
7     def panel(label, color, weight):
8         return ft.Container(
9             content=ft.Text(label, color="white"),
10            bgcolor=color,
11            padding=10,
12            expand=weight,
13            alignment=ft.alignment.center,
14        )
15
16     top_bar = panel("Header", "#102027", weight=False)
17     top_bar.height = 50
18
19     grid = ft.Column(
20         controls=[
21             top_bar,
22             ft.Row(
23                 controls=[
24                     panel("Nav", "#1c313a", weight=1),
25                     ft.Column(
26                         controls=[
27                             panel("Live Console", "#263238", weight
28                                 =3),
29                             panel("Status Bar", "#37474f", weight=1),
30                         ],
31                         expand=4,
32                         spacing=2,
33                     ),
34                 ],
35                 expand=True,
36                 spacing=2,
37             ),
38             expand=True,
39             spacing=2,
40         ]
41     )
42     page.add(grid)
43
44 ft.app(target=main)

```

This same code renders correctly whether:

- the window is maximized on a 4K monitor,
- resized to a narrow half-screen split,
- or embedded as a small docked panel inside another application.

Every panel resizes *relative to its siblings* — the nav column always stays narrower than the console, and the console always stays larger than the status bar, regardless of the absolute pixel dimensions of the window.

Takeaway

The shift from absolute positioning to Flet’s `Row/Column/expand` model mirrors the shift the web went through from table-based layouts to Flexbox and CSS Grid. By describing *relationships* between panels (“this sidebar is one-fourth the width of the main content”) instead of *coordinates* (“place this at x=250, y=0”), the resulting UI becomes inherently portable — full-screen, windowed, or docked, the interface matrix rebalances itself automatically.